Smart Healthcare

Ứng dụng đặt lịch khám kèm trợ lý AI đọc tờ hướng dẫn bệnh nhân

Báo cáo đồ án cuối kỳ

Công nghệ mới trong Công nghệ phần mềm 2026–2027

Thành viên	Module sở hữu
Trần Quốc Huy	authen, doctor
Võ Hữu Lộc	RAG
Nguyễn Chí Dũng	admin, booking

Ngày xuất bản: 27/10/2026

Mã nguồn: github.com/huulocc/smart-healthcare

Mục lục

Mục	Nội dung
1	Giới thiệu và phạm vi
2	Kiến trúc hệ thống
3	Luồng dữ liệu của một request AI
4	Ba năng lực AI đã làm
5	Chặn trước và kiểm sau: hai lớp an toàn
6	Đánh giá
7	Chi phí và độ trễ đo thực
8	Ba quyết định kiến trúc
9	Phân công và truy vết
10	Hạn chế và việc còn lại
11	Cách chạy

Báo cáo này được sinh ra bởi `docs/report/build_report.py`. Mọi con số trong phần đánh giá và chi phí đều được đọc trực tiếp từ các file kết quả trong `rag/eval/results/` lúc dựng file, không phải gõ tay vào bài viết.

1. Giới thiệu và phạm vi

Smart Healthcare là một ứng dụng đặt lịch khám cho phòng khám, kèm một trợ lý AI làm đúng hai việc: trả lời câu hỏi dựa trên các tờ hướng dẫn bệnh nhân, và gợi ý khoa khám từ triệu chứng người dùng mô tả. Sản phẩm gồm ba tiến trình do nhóm viết — giao diện React, backend Spring Boot, và một dịch vụ RAG viết bằng Python — cùng ba dịch vụ bên ngoài là SQL Server, Ollama và ChromaDB.

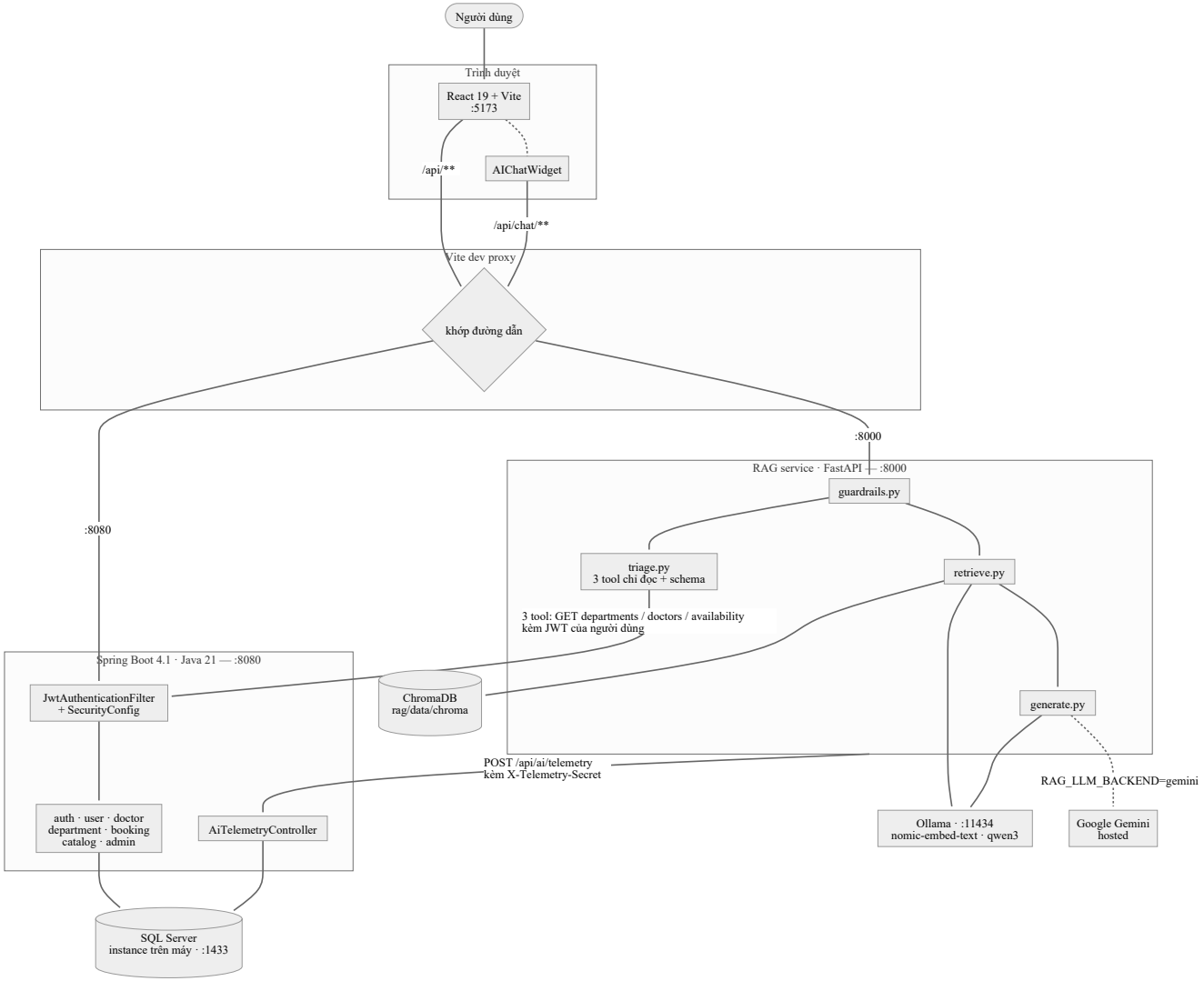
Việc trợ lý hoạt động trong lĩnh vực y tế định hình gần như mọi quyết định kỹ thuật trong báo cáo này. Một câu bịa về liều thuốc không phải lỗi vô hại, nên hệ thống được thiết kế theo hướng thà từ chối còn hơn đoán: câu trả lời nào không chứng minh được nguồn thì bị huỷ cả, không phải sửa cho vừa.

Phạm vi báo cáo

Báo cáo tập trung vào phần AI và những gì đo được, vì đó là phần đề yêu cầu đánh giá. Các module nghiệp vụ thông thường — đăng ký, đăng nhập, quản trị, đặt lịch — chỉ được nêu ở mức liên kết; chi tiết nằm trong `docs/TRACEABILITY.md` và `docs/OWNERSHIP.md`.

2. Kiến trúc hệ thống

Ba tiến trình do nhóm viết, ba dịch vụ bên ngoài. Trình duyệt luôn ở một origin duy nhất nhờ Vite proxy, nên hệ thống không cần cấu hình CORS.



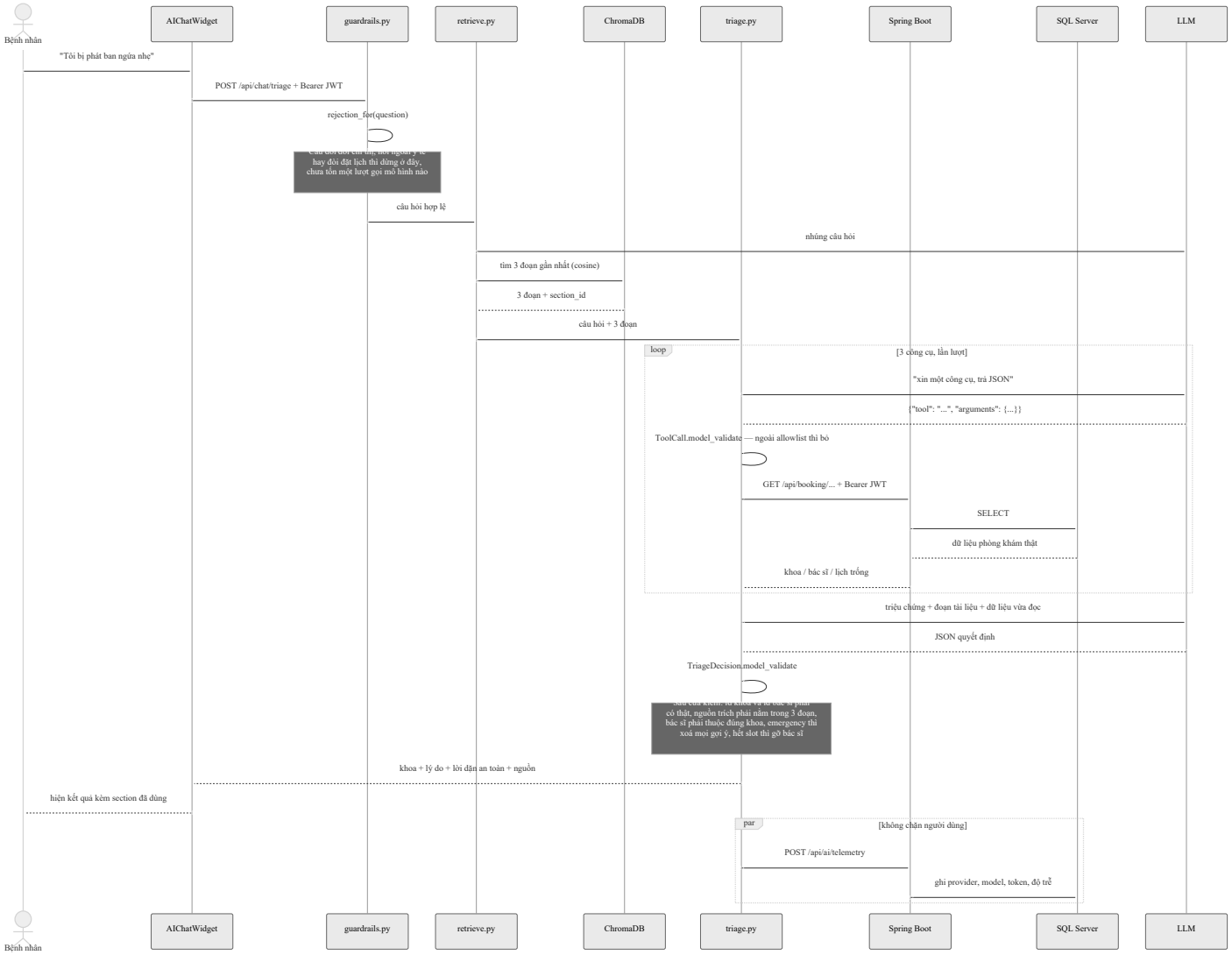
Hình 1 — Toàn cảnh hệ thống.

Ba điểm đáng nói

- Proxy quyết định theo thứ tự. `vite.config.ts` khai `/api/chat` trước `/api`, vì Vite khớp key theo thứ tự khai báo. Đảo lại thì mọi request chat đi nhầm sang Spring Boot và chatbot hỏng không một tiếng báo.
- Trợ lý không đọc database. Nó gọi lại đúng những endpoint mà giao diện gọi, mang theo JWT của chính người đang hỏi, nên không bao giờ thấy được nhiều hơn người đó được thấy.
- Hai nhà cung cấp LLM, chọn bằng biến môi trường. `RAG_LLM_BACKEND` chuyển giữa Ollama chạy local và Gemini. Phần nhúng giữ nguyên ở Ollama — index dựng bằng mô hình nào phải truy vấn bằng đúng mô hình đó, và `retrieve.py` dừng hẳn nếu phát hiện lệch.

3. Luồng dữ liệu của một request AI

Lấy luồng nặng nhất làm ví dụ: bệnh nhân mô tả triệu chứng và xin gợi ý khoa khám. Đây là luồng chạm vào cả năm thành phần: giao diện, backend, database, vector store và LLM.



Hình 2 — Một request gợi ý khoa khám, từ lúc gõ tới lúc ghi telemetry.

Bốn chỗ đáng chỉ ra

- Guardrail chặn trước khi tốn tiền. Bước 3 xảy ra trước mọi lệnh gọi mô hình. Một câu “bỏ qua chỉ thị phía trên” bị trả về ngay, không sinh token nào.
- Mô hình không bao giờ chạm database. Nó chỉ xin một công cụ bằng JSON; việc gọi thật do `triage.py` làm, qua Spring Boot, và chỉ ba hàm trong danh sách cho phép — cả ba đều `GET`.
- Kết quả bị kiểm trước khi tới người dùng. `TriageDecision` là schema Pydantic với `extra="forbid"`; sau khi parse còn sáu cửa kiểm nữa.
- Telemetry chạy song song và được phép hỏng. `rag/telemetry.py` nuốt mọi lỗi của chính nó — việc đo đạc không bao giờ được làm hỏng câu trả lời cho bệnh nhân.

4. Ba năng lực AI đã làm

Theo mục 4.1 của đề, nhóm ba người cần tối thiểu ba mục. Nhóm làm A, B, C.

	Năng lực	Ở đâu trong mã nguồn

A	RAG trên tài liệu: chia đoạn, nhúng, tìm theo vector, trả lời kèm trích nguồn, và nói rõ khi tài liệu không có câu trả lời	`rag/ingest.py`
		`rag/retrieve.py`
		`rag/generate.py`

B	Agent gọi công cụ: mô hình xin ba công cụ chỉ đọc để lấy dữ liệu thật từ Spring Boot	`rag/triage.py` — `_request_tool`
C	Structured output: kết quả là JSON kiểm theo schema Pydantic, và ứng dụng dùng nó để chặn sáu trường hợp mô hình bịa	`rag/triage.py` — `TriageDecision`

Nhóm không nhận mục D. Chức năng tải file lên có hoạt động — `rag/uploaded.py` trích chữ từ PDF hoặc TXT, nhúng, dùng cho đúng câu hỏi đó rồi quên — nhưng vì không ghi xuống đĩa và không ghi vào database, nó không đáp ứng vế "the result is stored and used by the app". Đây là lựa chọn có chủ đích về quyền riêng tư, và nhóm không tính nó là một năng lực đã làm.

5. Chặn trước và kiểm sau: hai lớp an toàn

An toàn trong hệ thống này không nằm ở một chỗ, mà ở hai lớp đặt trước và sau mô hình.

Lớp 1 — chặn trước khi gọi mô hình

`rag/guardrails.py` từ chối ba loại câu trước khi chạm tới mô hình: câu tìm cách đổi chỉ thị của trợ lý, câu hỏi ngoài phạm vi y tế, và câu đòi đặt hoặc huỷ lịch hẹn. Lớp này vừa là biện pháp an toàn vừa là biện pháp chi phí: năm case `boundary` trong bộ đánh giá tốn 0 lượt gọi mô hình.

Lớp 2 — sáu cửa kiểm sau khi mô hình trả lời

Sau khi JSON đã qua schema Pydantic, `triage.py` còn kiểm sáu điều. Không đạt một trong sáu thì cả câu trả lời bị huỷ và dịch vụ trả `503` — không có chuyện sửa cho vừa rồi gửi đi.

Cửa kiểm	Chặn điều gì
`department_id` phải có thật	mô hình bịa ra một khoa không tồn tại
`doctor_id` phải có thật	mô hình bịa ra một bác sĩ
bác sĩ thuộc đúng khoa	ghép bác sĩ của khoa khác vào gợi ý
nguồn trích nằm trong 3 đoạn	trích một section không hề được truy hồi
`emergency` thì xoá mọi gợi ý	đẩy người cần cấp cứu vào luồng đặt lịch
hết slot thì gỡ bác sĩ	gợi ý một bác sĩ không còn lịch

Cách làm này có giá của nó, và mục 6 cho thấy giá đó bằng bao nhiêu: cửa kiểm nguồn trích từng đánh hỏng 6 trên 25 case, không phải vì mô hình trả lời sai mà vì nó trích nguồn sai định dạng.

6. Đánh giá

Nhóm đo hai thứ tách biệt: truy hồi đúng hay sai (có lấy đúng đoạn tài liệu không), và hành vi đầu cuối của trợ lý qua HTTP thật.

6.1 Truy hồi

Bộ `rag/eval/eval_questions.yaml` chấm 60 câu có đáp án, trên corpus 33 đoạn. Ground truth trỏ tới section id chứ không phải số thứ tự chunk, nên bộ đề vẫn dùng được khi đổi cách chia đoạn.

Nhóm câu hỏi	Số câu	recall@1	recall@3
`factual`	17	94.1%	94.1%
`numeric`	11	90.9%	100.0%
`paraphrase`	8	62.5%	75.0%
`patient-voice`	7	57.1%	71.4%
`synthesis`	5	60.0%	80.0%
`safety`	8	87.5%	100.0%
`disambiguation`	4	100.0%	100.0%
Tổng	60	81.7%	90.0%

Phân bố này mới là thông tin hữu ích, không phải con số tổng. Câu `factual` và `numeric` dùng chung từ vựng với tài liệu nên gần như bão hoà, trong khi `paraphrase` và `patient-voice` — tức cách một bệnh nhân thật sự hỏi — là chỗ mô hình nhúng thực sự yếu. Đó là hướng cải thiện rõ ràng nhất nếu dự án đi tiếp.

6.2 Hành vi đầu cuối: 25 case qua HTTP thật

Bộ `rag/eval/chatbot_cases.json` gọi dịch vụ đang chạy, với backend và database thật. Script từ chối chạy nếu bộ case dưới 20 case — một chốt chống việc chọn vài case dễ rồi gọi đó là một lượt đánh giá.

Lượt đầu đạt 76.0%, và con số đó dẫn tới một lỗi thật. Sáu case fail đều cùng một lý do: `The recommendation cites an unknown source`. Dò lại thì mô hình trả về `source_ids = ["2", "3"]` — tức số trong ngoặc vuông của context, không phải section id. Prompt v1 chỉ nói "section IDs supplied in leaflet_context", mà trong context có đúng hai thứ đều hợp lý gọi là id.

Quan trọng hơn: những case đó chẩn đoán đúng — đúng khoa, đúng mức độ khẩn, bác sĩ đúng khoa — và bị huỷ chỉ vì định dạng trích dẫn. Sửa là một câu trong `rag/prompts/triage_v2.txt`:

> *"Every entry in source_ids must be copied verbatim from a (section: ...) label in leaflet_context — the whole slash-separated string. Never put the bracket number such as "2" in source_ids."*

Nhóm case	prompt v1	prompt v2
`boundary`	5/5	5/5
`guidance`	2/5	5/5
`guidance_safety`	0/2	2/2

Nhóm case	prompt v1	prompt v2
`leaflet`	7/7	7/7
`upload`	2/3	3/3
`upload_boundary`	3/3	3/3
Tổng	19/25 = 76.0%	25/25 = 100.0%

Cùng bộ case (cùng `case_set_sha256`), cùng mô hình (`embed=ollama:nomic-embed-text llm=ollama:qwen3 think=True`), khác đúng một câu trong prompt.

6.3 Chạy lại cùng cấu hình thì không ra cùng kết quả

Nhóm chạy prompt v2 hai lần, cùng máy, cùng mô hình, cùng bộ case. Lần đầu 25/25; lần hai 24/25, với `G07` trượt đúng cửa kiểm nguồn trích mà prompt v2 sinh ra để sửa.

Điều này phải được nói thẳng thay vì báo con số đẹp hơn. Nhiệt độ đặt 0 và phần truy hồi là tất định, nên biến thiên nằm ở chính mô hình sinh văn bản. Nói cách khác: prompt v2 giảm mạnh lỗi trích nguồn sai chứ không xoá hẳn nó, và với hai lượt chạy thì chưa đủ mẫu để nói tỉ lệ còn lại là bao nhiêu. Muốn chắc thì phải chạy nhiều lượt và báo khoảng dao động, không phải một con số.

6.4 Con số 100% này không có nghĩa là trợ lý trả lời đúng y khoa

Chính file kết quả ghi dòng này:

> *"Automatic assertions are a screen; manually review medical safety and source grounding in the saved answers."*

Hai điều nữa phải nói rõ. Một, hai case `G01` và `G03` pass với `source_ids = []` — tập rỗng là tập con của mọi tập, nên một câu trả lời không trích nguồn nào lọt qua dễ hơn một câu trích sai. Đây là lỗ thật trong cửa kiểm, đã có từ trước. Hai, case `G04` trích trùng một section hai lần.

7. Chi phí và độ trễ đo thực

Trang quản trị có bảng thống kê mức dùng AI. Dịch vụ RAG gửi mỗi lượt gọi về `POST /api/ai/telemetry` kèm một secret dùng chung; backend từ chối nếu secret không khớp, và dịch vụ RAG tự tắt telemetry nếu chưa cấu hình — cả hai phía đều fail closed.

Số dưới đây lấy từ một lượt chạy 25 case trên máy cá nhân, Ollama, không dùng GPU chuyên dụng.

Provider / model	Sự kiện	Độ trễ nhỏ nhất	Lớn nhất	Trung bình	Chi phí
`OLLAMA qwen3`	10	27.5s	92.5s	52.6s	0
`OLLAMA qwen3`	8	162.4s	423.3s	239.5s	0
`GOOGLE gemini-3.5-flash`	1	97.5s	97.5s	97.5s	$0.000003

Tổng cộng 19 sự kiện, 12,799 token vào và 1,288 token ra, chi phí $0.0000 — toàn bộ suy luận chạy local.

Đánh đổi giữa hai nhà cung cấp

Ollama miễn phí và riêng tư nhưng chậm. Gemini nhanh hơn một bậc, nhưng hạn mức miễn phí không gánh nổi một lượt đánh giá đầy đủ: bộ 25 case tốn 44 lượt gọi mô hình (mỗi case triage tốn 4), và hạn mức cạn sau khoảng 15–20 lượt. Nhóm đã thử hai khóa khác nhau, cả hai đều dừng giữa chừng; bằng chứng lưu ở `rag/eval/results/chatbot-gemini-quota-hit.json`. Kết luận vận hành: đánh giá chạy bằng Ollama, Gemini để cho lúc demo.

Một cảnh báo về dữ liệu

Bảng `ai_usage_events` có sẵn 14 dòng dữ liệu demo do `backend/sqlserver/admin/002_admin_features.sql` chèn vào, kèm cả cột chi phí được viết tay. Script `docs/report/collect_telemetry.py` loại chúng ra trước khi tính, nên mọi số ở trên đều là đo thật.

8. Ba quyết định kiến trúc

Ba quyết định dưới đây được ghi đầy đủ trong `docs/adr/`, mỗi quyết định kèm các phương án đã cân nhắc và chỗ kiểm chứng được trong mã nguồn.

ADR	Quyết định	Cái giá phải trả
0001	Không dùng ngưỡng similarity để từ chối. Trên bộ đánh giá, câu không trả lời được (Q25) đạt 0.607, cao hơn cả đoạn đúng của một câu trả lời được (Q14, 0.584). Không có ngưỡng nào đặt giữa hai số đó mà không làm hỏng một trong hai việc.	Việc từ chối trở thành hành vi của mô hình, tức không tất định — phải có nhóm `refusal` trong bộ đánh giá để biết nó còn chạy hay không.
0002	SQL Server cài trên máy, xoá hẳn compose file. Database của nhóm lưu hành dưới dạng file `.bak`, mà một container sẽ khởi động rỗng và sống tách biệt với chính file đang được chuyền tay.	Không còn lệnh khởi động một dòng. Đây là lý do phần "Cách chạy" trong `README.md` dài hơn hẳn một dự án có Docker.
0003	Vai trò nằm trong claim của JWT, không tra database mỗi request. Bỏ một truy vấn trên mọi request được xác thực.	Thay đổi vai trò có độ trễ. Admin duyệt một người thành bác sĩ thì người đó phải đăng nhập lại; cửa sổ lệch tối đa bằng tuổi thọ token, 30 ngày với "remember me".

9. Phân công và truy vết

Mỗi thành viên sở hữu ít nhất một module và chịu trách nhiệm cho toàn bộ backend, frontend và test của module đó. Bảng đầy đủ ở `docs/OWNERSHIP.md`.

Thành viên	Module sở hữu	User story
Trần Quốc Huy	authen, doctor	`US-AU-01`…`03`, `US-DR-01`…`05`
Võ Hữu Lộc	RAG	`US-AI-01`, `US-AI-02`
Nguyễn Chí Dũng	admin, booking	`US-AD-01`, `US-AD-02`, `US-BK-01`, `US-BK-02`

`docs/TRACEABILITY.md` nối 14 user story tới endpoint, file mã nguồn và test cụ thể, nên mỗi yêu cầu đều chỉ ra được chỗ nó được thực hiện và chỗ nó được giữ.

Tầng	Kiểm thử
Backend (JUnit)	165 test trên 19 file
RAG (unittest)	36 test
Frontend	3 bộ test: booking, doctor, aiUsage
Đánh giá AI	60 câu truy hồi + 25 case đầu cuối

10. Hạn chế và việc còn lại

Phần này liệt kê những gì chưa làm được hoặc chưa chứng minh được, vì một báo cáo chỉ nói phần tốt thì không dùng để ra quyết định được.

Về chất lượng AI

- Truy hồi yếu ở đúng chỗ quan trọng nhất. `patient-voice` chỉ 57.1% recall@1 và `paraphrase` 62.5% — đó lại chính là cách bệnh nhân thật hỏi.
- Cửa kiểm nguồn trích nhận cả tập rỗng. Một câu trả lời không trích nguồn nào vẫn pass. Cần thêm ràng buộc "ít nhất một nguồn".
- 100% là điểm của phép kiểm tự động, không phải của độ đúng y khoa. Phải đọc tay mới kết luận được về an toàn.

Về sản phẩm

- Không có Docker. Người mới phải tự cài SQL Server và bật TCP/IP.
- Suy luận local chậm, và luồng gợi ý khoa chậm hơn hẳn. Trả lời từ tờ hướng dẫn trung bình 53 giây, còn một lượt triage trung bình 240 giây và có lần tới 423 giây, vì mỗi lượt là bốn lần gọi mô hình nối tiếp nhau. Chấp nhận được cho đồ án, không chấp nhận được cho một sản phẩm thật.
- Kết quả không lặp lại được chính xác. Hai lượt chạy cùng cấu hình cho hai kết quả khác nhau (mục 6.3). Mọi con số đầu cuối trong báo cáo này nên được đọc như một mẫu, không phải một hằng số.
- File tải lên không được lưu, nên năng lực D của đề không được nhận.

Về tài liệu

- `AI_USAGE.md` còn thiếu. Mục 2.1 chưa viết, và bốn mảng việc đã merge sau 24/10 chưa có mục riêng — trong đó có PR #11/#13 là phần dựng nên chính năng lực B và C.

11. Cách chạy

Thứ tự bật: SQL Server → Ollama → backend → `ingest.py` → RAG service → frontend. `ingest.py` phải chạy trước `server.py` ít nhất một lần, nếu không mọi câu hỏi dừng ở `No index found`.

Thành phần	Cổng	Khởi động bằng
SQL Server	1433	dịch vụ Windows
Ollama	11434	`ollama serve`
Spring Boot	8080	`.\mvnw.cmd spring-boot:run`
RAG service	8000	`python rag/server.py`
Frontend (Vite)	5173	`npm run dev`

Một cái bẫy đáng biết trước: mặc định trong `rag/config.py` là Gemini, không phải Ollama. Một bản `rag/.env` vừa chép từ file mẫu sẽ đi đường Gemini với khoá rỗng và hỏng ngay câu đầu. Muốn chạy local thì phải đặt `RAG_LLM_BACKEND=ollama`.

Tài liệu chi tiết

File	Nội dung
`README.md`	cách chạy, biến môi trường, tài khoản test
`docs/ARCHITECTURE.md`	hai sơ đồ trong báo cáo này, dạng mermaid
`docs/adr/`	ba quyết định kiến trúc
`docs/USER_STORIES.md`	14 story kèm tiêu chí chấp nhận
`docs/TRACEABILITY.md`	story → endpoint → file → test
`docs/CHATBOT.md`	trợ lý hoạt động thế nào
`AI_USAGE.md`	nhật ký dùng công cụ AI